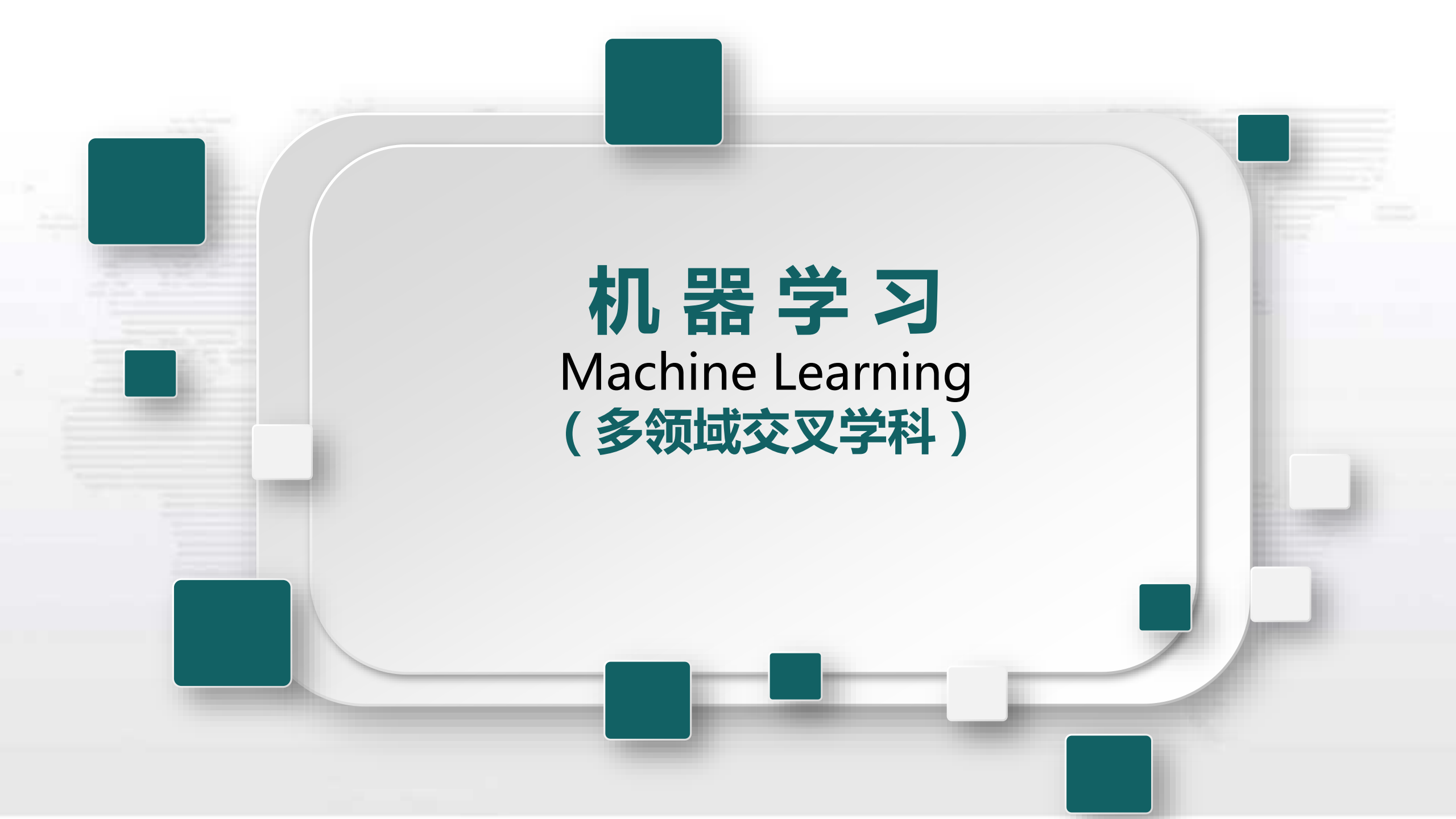




机器学习

Machine Learning
(多领域交叉学科)



目录

CONTENTS

1

机器学习简介

2

机器学习相关基础知识

3

K近邻算法 (KNN)

4

线性回归法

5

主成分分析法 (PCA)

6

逻辑回归法

7

支持向量机算法 (SVM)

8

决策树算法

9

集成学习和随机森林

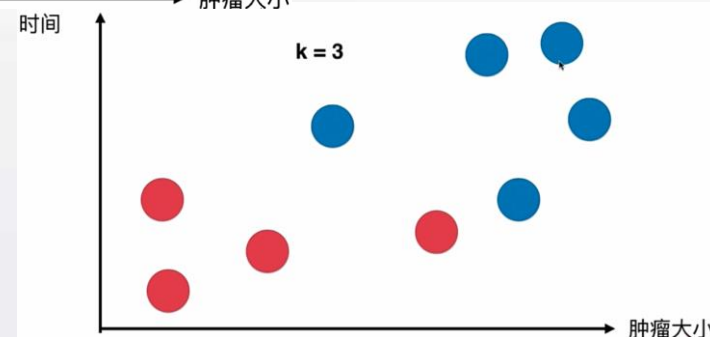
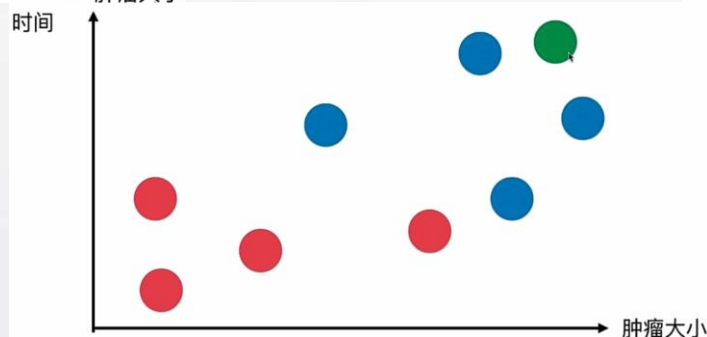
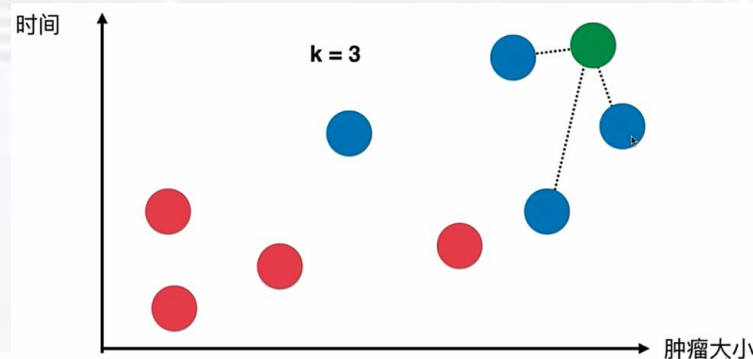
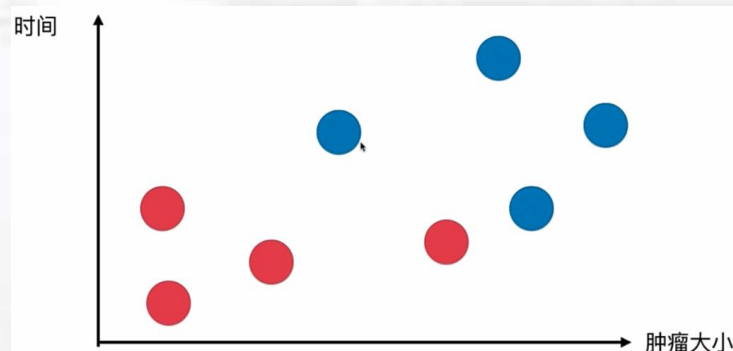
1

K近邻算法基础

- 思想极度简单
- 应用数学知识少（近乎为0）
- 效果好（也有缺点）
- 可以解释机器学习算法使用过程中的很多细节问题
- 更完整的刻画机器学习应用的流程

1

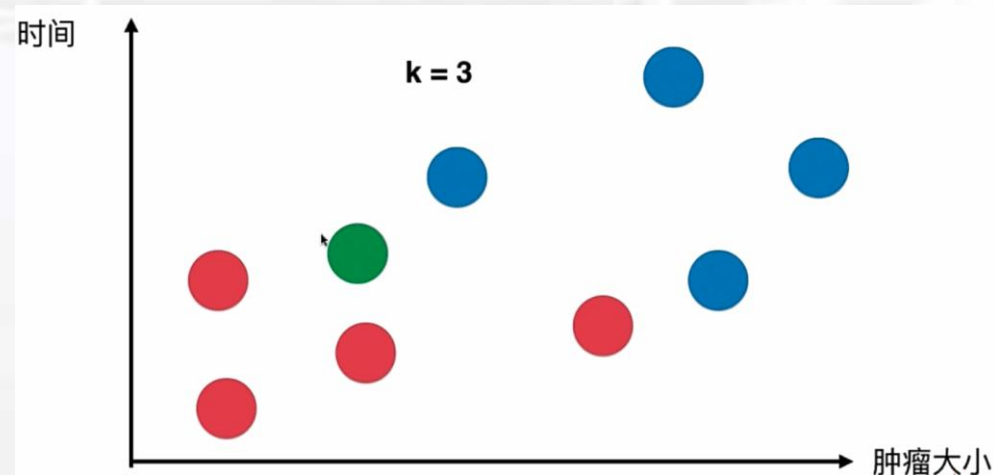
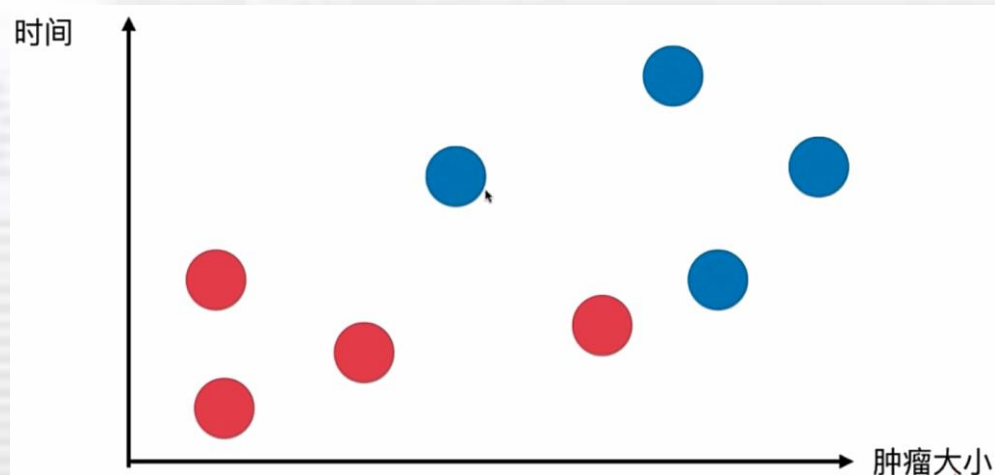
K近邻算法基础



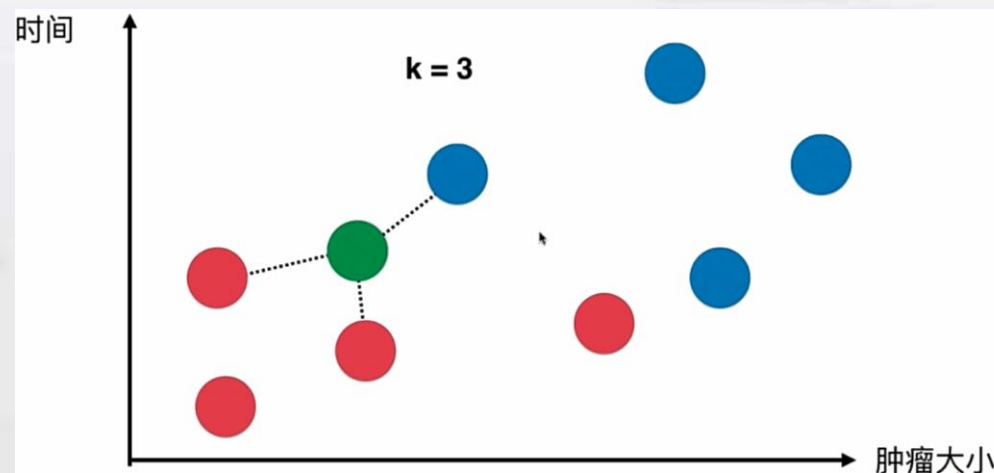
- 1、初始信息：数据点是分布在一个特征空间中的，此处是降维以后的效果，横轴是发现肿块的大小，纵轴是发现肿块的时间，对于这个点，已经知道它是恶性的还是良性的肿瘤。其中蓝色表示恶性，红色表示良性。
- 2、若有一个新的肿瘤数据点，如何判断是良性或恶性？
- 3、 $K=3$ （此处先可理解为根据经验取得的一个最好的值）指找离新加入点最近的3个点，这3个点以它们自己的标签进行投票，目前看，蓝色：红色=3:0，所以新的点大概率是蓝色点
- 4、K近邻的本质：两个样本，如果足够的相似的话，那么就有更高的概率属于同一个类别（这里看一个样本是不够的，需要看K个样本，哪一个样本最多，则认为有可能属于哪个类别）
- 5、相似性：是用特征空间中两点间的距离进行描述的

1

K近邻算法基础



- 1、选择3个最近的点
- 2、 $K=3$ ，蓝色：红色=2:1，所以新的点大概率是红色点
- 3、K近邻算法，首先解决的是监督学习中的分类问题，它也可以解决回归问题，后续会介绍
- 4、下面看一下KNN的编程实现。



1

K近邻算法基础

kNN 基础

```
In [1]: 1 import numpy as np
        2 import matplotlib.pyplot as plt
```

```
In [2]: 1 raw_data_X = [[3.393533211, 2.331273381],
        2               [3.110073483, 1.781539638],
        3               [1.343808831, 3.368360954],
        4               [3.582294042, 4.679179110],
        5               [2.280362439, 2.866990263],
        6               [7.423436942, 4.696522875],
        7               [5.745051997, 3.533989803],
        8               [9.172168622, 2.511101045],
        9               [7.792783481, 3.424088941],
       10              [7.939820817, 0.791637231]
       11              ]
       12 raw_data_y = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
```

```
In [ ]: 1 X_train = np.array(raw_data_X)
        2 y_train = np.array(raw_data_y)
```

这里，我们将所有的数据都作为训练集
其中：

X_train: 样本的特殊值

y_train: 样本的标签label

1

K近邻算法基础

```
In [4]: 1 X_train
```

```
Out[4]: array([[ 3.39353321,  2.33127338],
                [ 3.11007348,  1.78153964],
                [ 1.34380883,  3.36836095],
                [ 3.58229404,  4.67917911],
                [ 2.28036244,  2.86699026],
                [ 7.42343694,  4.69652288],
                [ 5.745052   ,  3.5339898 ],
                [ 9.17216862,  2.51110105],
                [ 7.79278348,  3.42408894],
                [ 7.93982082,  0.79163723]])
```

```
In [5]: 1 y_train
```

```
Out[5]: array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1])
```

X_train: 样本的特殊值，是一个2维的矩阵

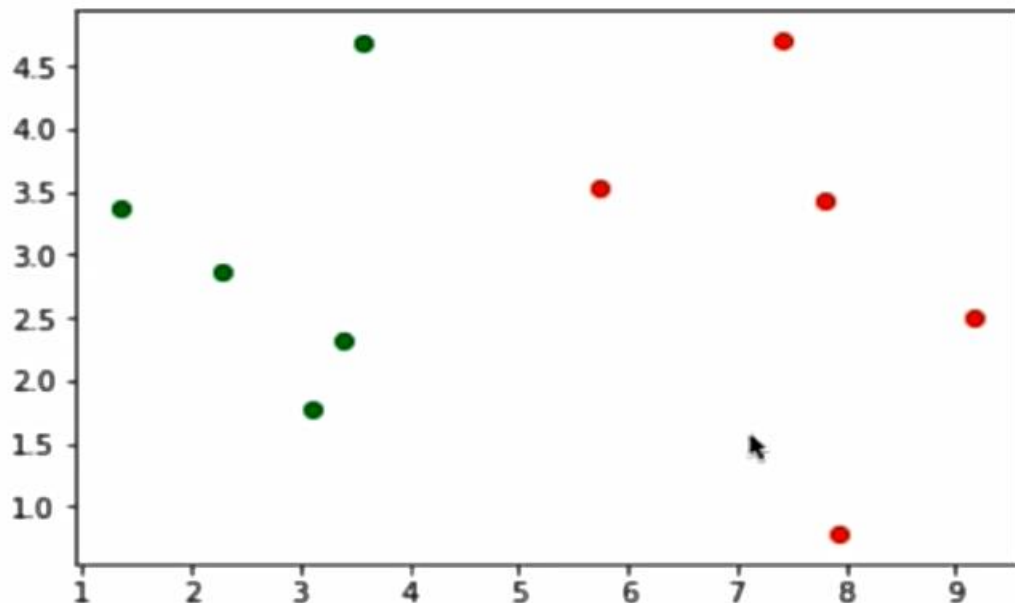
y_train: 样本的标签label，是一个1维的向量

1

K近邻算法基础

In [6]:

```
1 plt.scatter(X_train[y_train==0,0], X_train[y_train==0,1], color='g')  
2 plt.scatter(X_train[y_train==1,0], X_train[y_train==1,1], color='r')  
3 plt.show()
```



散点图的绘制，与鸢尾花数据的绘制是一样的，绘制每一个点在特征平面的位置
对于 $y_{train}=0$ ，取出第0列和第1列，颜色设置为绿色
对于 $y_{train}=1$ ，取出第0列和第1列，颜色设置为红色

1

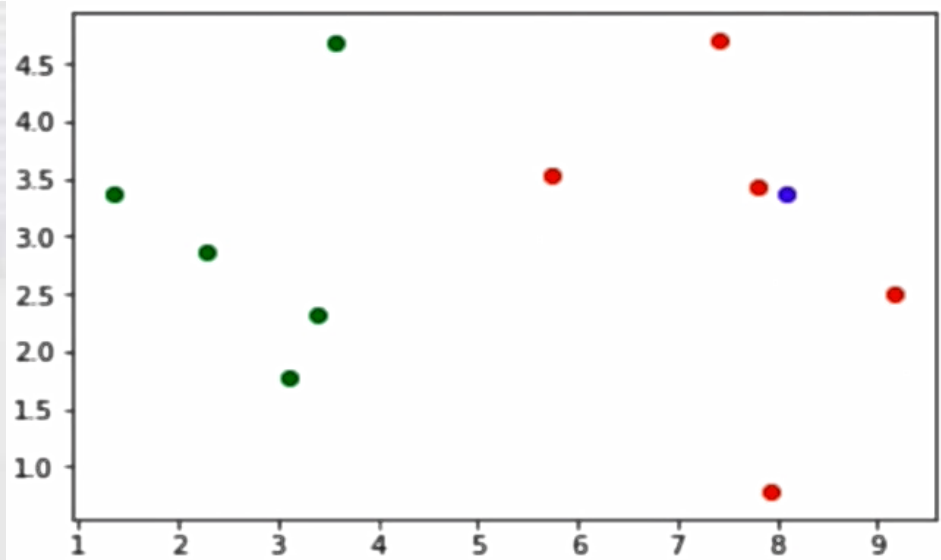
K近邻算法基础

假如来了一组新的数据，预测它属于哪一类，是红色还是绿色，或良性肿瘤还是恶性肿瘤

```
In [7]: 1 x = np.array([8.093607318, 3.365731514])
```

为了清楚表达，对新加入的点单独进行绘制

```
In [ ]: 1 plt.scatter(X_train[y_train==0,0], X_train[y_train==0,1], color='g')  
2 plt.scatter(X_train[y_train==1,0], X_train[y_train==1,1], color='r')  
3 plt.scatter(x[0], x[1], color='b')  
4 plt.show()
```



1

K近邻算法基础

在X_train中取出小x_train，相应的都计算一个新的距离

kNN的过程

```
In [11]: 1 from math import sqrt
          2 distances = []
          3 for x_train in X_train:
          4     d = sqrt(np.sum((x_train - x)**2))
          5     distances.append(d)
```

```
In [12]: 1 distances
```

```
Out[12]: [4.812566907609877,
           5.229270827235305,
           6.749798999160064,
           4.6986266144110695,
           5.83460014556857,
           1.4900114024329525,
           2.354574897431513,
           1.3761132675144652,
           0.3064319992975,
           2.5786840957478887]
```

欧氏距离

计算两点之间的距离，指对应维度相应特征值的差的平方和，再求平方根

$$\sqrt{(x^{(a)} - x^{(b)})^2 + (y^{(a)} - y^{(b)})^2}$$

$$\sqrt{(x^{(a)} - x^{(b)})^2 + (y^{(a)} - y^{(b)})^2 + (z^{(a)} - z^{(b)})^2}$$

$$\sqrt{(X_1^{(a)} - X_1^{(b)})^2 + (X_2^{(a)} - X_2^{(b)})^2 + \dots + (X_n^{(a)} - X_n^{(b)})^2}$$

$$\sqrt{\sum_{i=1}^n (X_i^{(a)} - X_i^{(b)})^2}$$

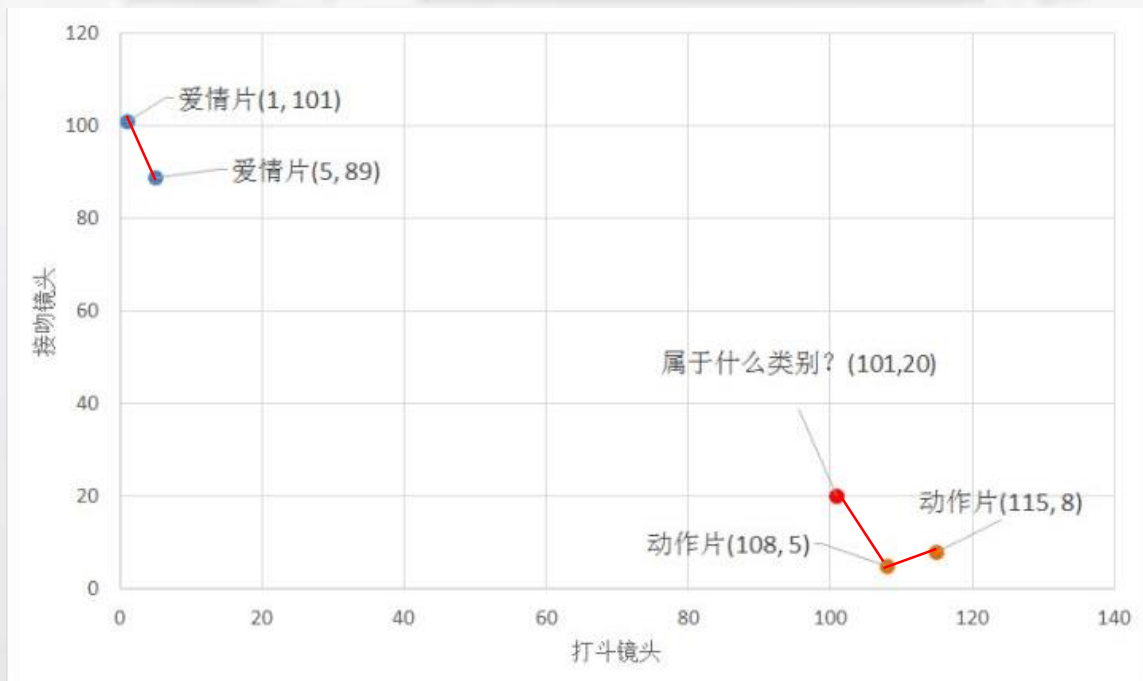
1

K近邻算法基础

欧氏距离(Euclidean distance)

$$d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

电影分类

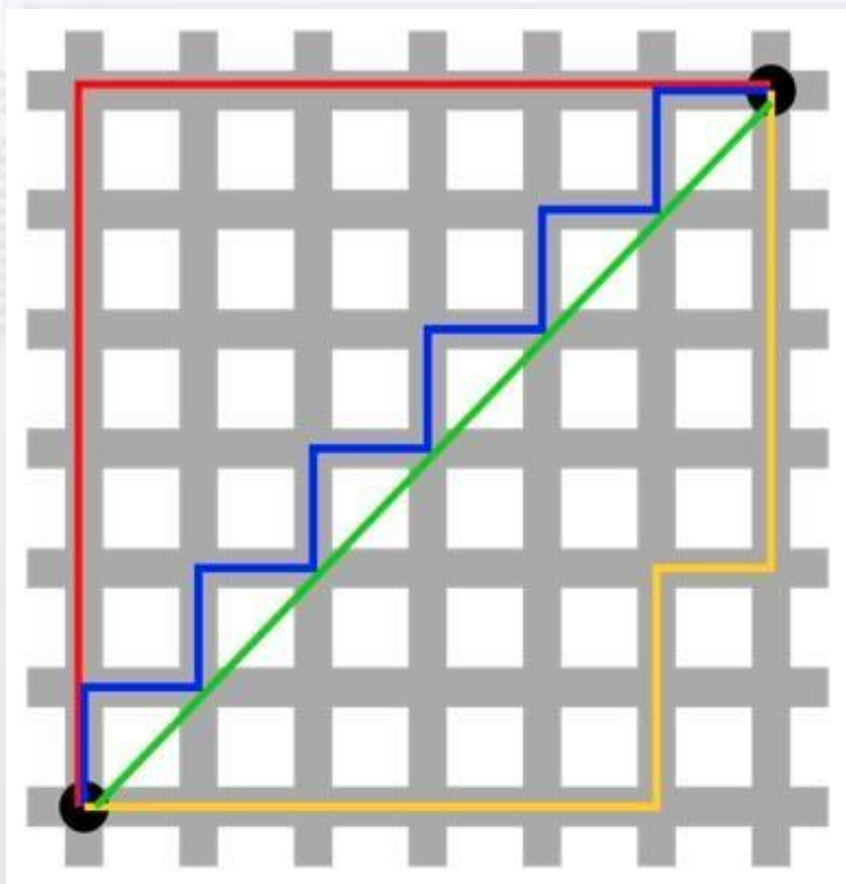


欧几里得度量 (Euclidean Metric) (也称欧氏距离) 是一个通常采用的距离定义，指在 m 维空间中两个点之间的真实距离，或者向量的自然长度 (即该点到原点的距离)。在二维和三维空间中的欧氏距离就是两点之间的实际距离。

1

K近邻算法基础

曼哈顿距离(Manhattan distance)



$$d(x, y) = \sum_i |x_i - y_i|$$

想象你在城市道路里，要从一个十字路口开车到另外一个十字路口，驾驶距离是两点间的直线距离吗？显然不是，除非你能穿越大楼。实际驾驶距离就是这个“曼哈顿距离”。而这也是曼哈顿距离名称的来源，曼哈顿距离也称为城市街区距离(City Block distance)。

1

K近邻算法基础

切比雪夫距离(Chebyshev distance)

$$d(x, y) = \max_i |x_i - y_i|$$

二个点之间的距离定义是其各坐标数值差绝对值的最大值。

国际象棋棋盘上二个位置间的切比雪夫距离是指王要从一个位子移至另一个位子需要走的步数。由于王可以往斜前或斜后方向移动一格，因此可以较有效率的到达目的的格子。上图是棋盘上所有位置距f6位置的切比雪夫距离。

	a	b	c	d	e	f	g	h	
8	5	4	3	2	2	2	2	2	8
7	5	4	3	2	1	1	1	2	7
6	5	4	3	2	1		1	2	6
5	5	4	3	2	1	1	1	2	5
4	5	4	3	2	2	2	2	2	4
3	5	4	3	3	3	3	3	3	3
2	5	4	4	4	4	4	4	4	2
1	5	5	5	5	5	5	5	5	1
	a	b	c	d	e	f	g	h	

1

K近邻算法基础

闵可夫斯基距离(Minkowski distance)

p 取1或2时的闵氏距离是最为常用的

$p = 2$ 即为欧氏距离，

$p = 1$ 时则为曼哈顿距离。

当 p 取无穷时的极限情况下，可以得到切比雪夫距离

$$d(x, y) = \left(\sum_i |x_i - y_i|^p \right)^{\frac{1}{p}}$$

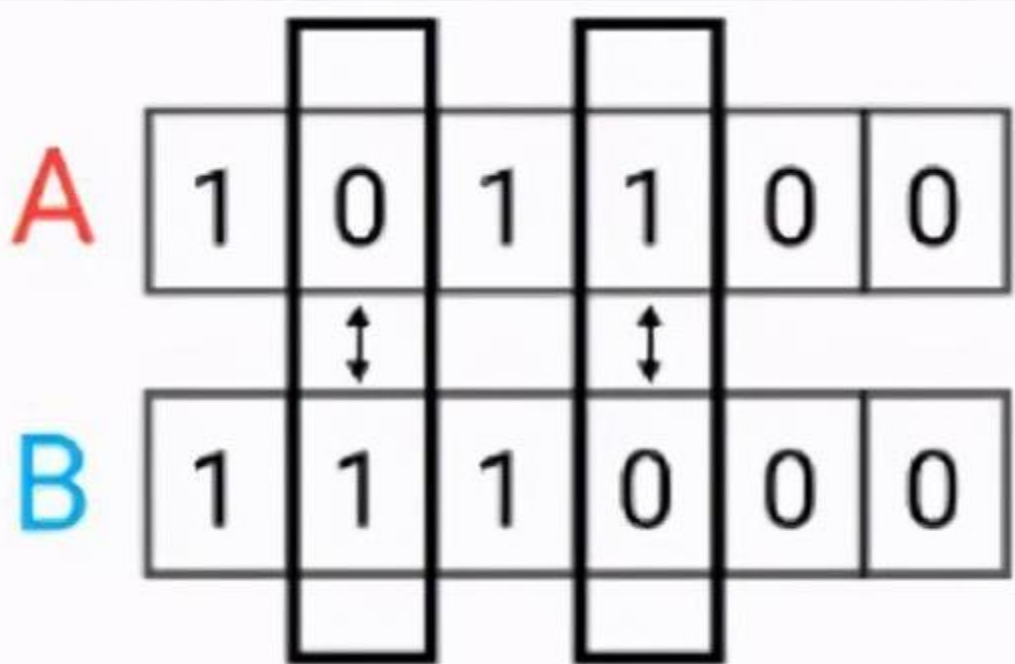
1

K近邻算法基础

汉明距离(Hamming distance)

$$d(x, y) = \frac{1}{N} \sum_i 1_{x_i \neq y_i}$$

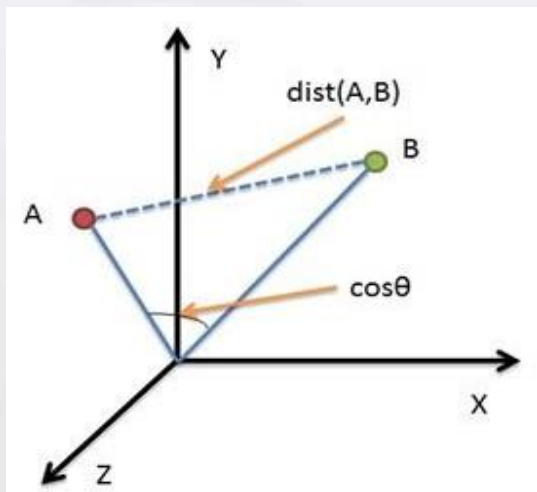
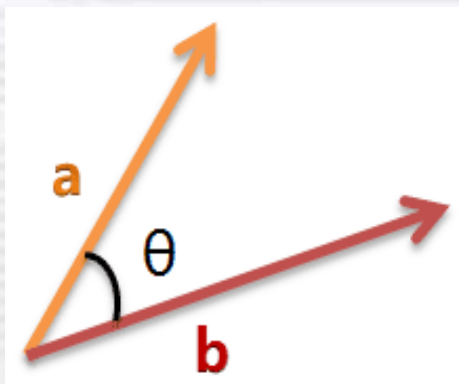
汉明距离是使用在数据传输差错控制编码里面的，汉明距离是一个概念，它表示两个（相同长度）字对应位不同的数量，我们以表示两个字之间的汉明距离。对两个字符串进行异或运算，并统计结果为1的个数，那么这个数就是汉明距离。



1

K近邻算法基础

余弦相似度



两个向量有相同的指向时，余弦相似度的值为1；两个向量夹角为 90° 时，余弦相似度的值为0；两个向量指向完全相反的方向时，余弦相似度的值为-1。

假定 A 和 B 是两个 n 维向量， A 是 $[A_1, A_2, \dots, A_n]$ ， B 是 $[B_1, B_2, \dots, B_n]$ ，则 A 和 B 的夹角的余弦等于：

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}}$$

1

K近邻算法基础

除了使用for循环，还可以直接用列表

```
In [13]: 1 distances = [sqrt(np.sum((x_train - x)**2)) for x_train in X_train]
```

```
In [14]: 1 distances
```

```
Out[14]: [4.812566907609877,  
          5.229270827235305,  
          6.749798999160064,  
          4.6986266144110695,  
          5.83460014556857,  
          1.4900114024329525,  
          2.354574897431513,  
          1.3761132675144652,  
          0.3064319992975,  
          2.5786840957478887]
```

判断离新加入的点距离X_train最近的点是哪些点？

1

K近邻算法基础

对所求得距离数值进行排序后，可得到最小距离或次小距离，但这些距离的值是没有意义的，**问题的关键**：是哪些点与新加入的点距离最小或次小？由此需要获取这些点的索引位置。
采取方法：调用`argsort()`函数

```
In [15]: 1 np.argsort(distances)
```

```
Out[15]: array([8, 7, 5, 6, 9, 3, 0, 1, 4, 2])
```

```
In [16]: 1 nearest = np.argsort(distances)
```

```
In [17]: 1 k = 6
```

```
In [18]: 1 topK_y = [y_train[i] for i in nearest[:k]]
```

```
In [19]: 1 topK_y
```

```
Out[19]: [1, 1, 1, 1, 1, 0]
```

k=6 表示取6个点

在最近距离列表nearest中，取前k个点，
求出这些点的y_train值

1

K近邻算法基础

如何求不同类别点的个数？

调用collections库中的Counter类

在Counter()中传入一个数组，得到的结果是，这个数组中的元素和元素出现的频次进行的统计数据，表现为{键：值}对的字典形式

```
In [20]: 1 from collections import Counter  
        2 Counter(topK_y)
```

```
Out[20]: Counter({0: 1, 1: 5})
```

0元素有1个，1元素有5个

```
In [21]: 1 votes = Counter(topK_y)
```

```
In [24]: 1 votes.most_common(2)
```

```
Out[24]: [(1, 5), (0, 1)]
```

```
In [25]: 1 votes.most_common(1)
```

```
Out[25]: [(1, 5)]
```

调用Counter类中的most_common()，获取票数最多的n个元素，返回的是一个列表，里面的元素由元组构成

这里票数最多的1个元素

```
In [26]: 1 votes.most_common(1)[0][0]
```

```
Out[26]: 1
```

KNN算法的预测结果为1，即最终投票，票数最多的类别是1，表示恶性肿瘤的可能性更大一些

1

K近邻算法基础

```
In [20]: 1 from collections import Counter  
        2 Counter(topK_y)
```

```
Out[20]: Counter({0: 1, 1: 5})
```

```
In [21]: 1 votes = Counter(topK_y)
```

```
In [24]: 1 votes.most_common(2)
```

```
Out[24]: [(1, 5), (0, 1)]
```

```
In [25]: 1 votes.most_common(1)
```

```
Out[25]: [(1, 5)]
```

```
In [26]: 1 votes.most_common(1)[0][0]
```

```
Out[26]: 1
```

```
In [27]: 1 predict_y = votes.most_common(1)[0][0]
```

```
In [28]: 1 predict_y
```

```
Out[28]: 1
```

0元素有1个，1元素有5个

调用Counter类中的most_common()，获取票数最多的n个元素，返回的是一个列表，里面的元素由元组构成

这里票数最多的1个元素

KNN算法的预测结果为1，即最终投票，票数最多的类别是1，表示恶性肿瘤的可能性更大一些

2

机器学习算法封装

2.1 以函数的形式，将KNN算法封装

```
1 import numpy as np
2 from math import sqrt
3 from collections import Counter
4
5
6 def knn_classify(k, X_train, y_train, x):
7
8     assert 1 <= k <= X_train.shape[0], "k must be valid"
9     assert X_train.shape[0] == y_train.shape[0], \
10         "the size of X_train must equal to the size of y_train"
11     assert X_train.shape[1] == x.shape[0], \
12         "the feature number of x must be equal to X_train"
13
14     distances = [sqrt(np.sum((x_train - x)**2)) for x_train in X_train]
15     nearest = np.argsort(distances)
16
17     topK_y = [y_train[i] for i in nearest[:k]]
18     votes = Counter(topK_y)
19
20     return votes.most_common(1)[0][0]
21
```


2

机器学习算法封装

```
In [1]: 1 import numpy as np
        2 import matplotlib.pyplot as plt
        3
        4 raw_data_X = [[3.393533211, 2.331273381],
        5              [3.110073483, 1.781539638],
        6              [1.343808831, 3.368360954],
        7              [3.582294042, 4.679179110],
        8              [2.280362439, 2.866990263],
        9              [7.423436942, 4.696522875],
       10              [5.745051997, 3.533989803],
       11              [9.172168622, 2.511101045],
       12              [7.792783481, 3.424088941],
       13              [7.939820817, 0.791637231]
       14              ]
       15 raw_data_y = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
       16
       17 X_train = np.array(raw_data_X)
       18 y_train = np.array(raw_data_y)
       19
       20 x = np.array([8.093607318, 3.365731514])
```

```
In [2]: 1 %run kNN_function/kNN.py
```

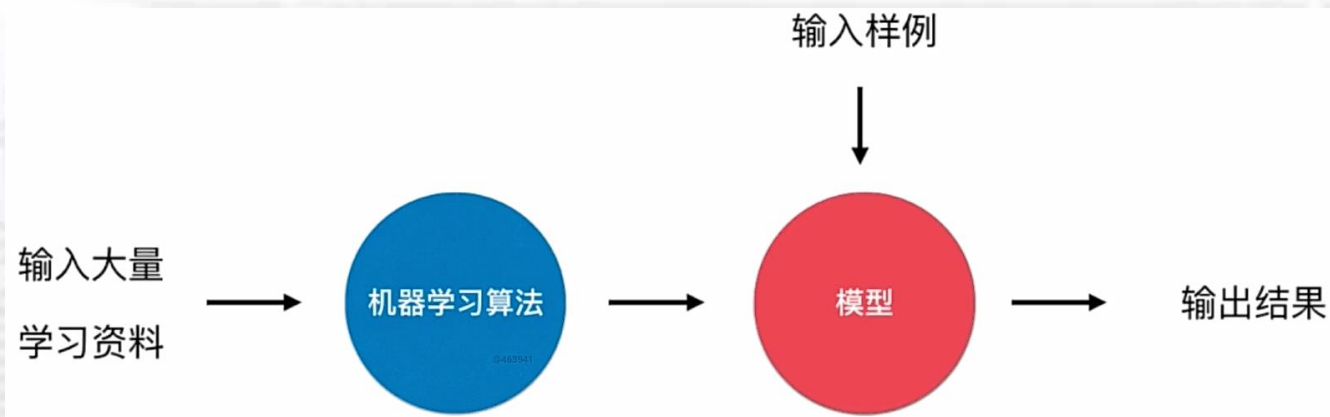
```
In [3]: 1 predict_y = kNN_classify(6, X_train, y_train, x)
```

```
In [4]: 1 predict_y
```

```
Out[4]: 1
```

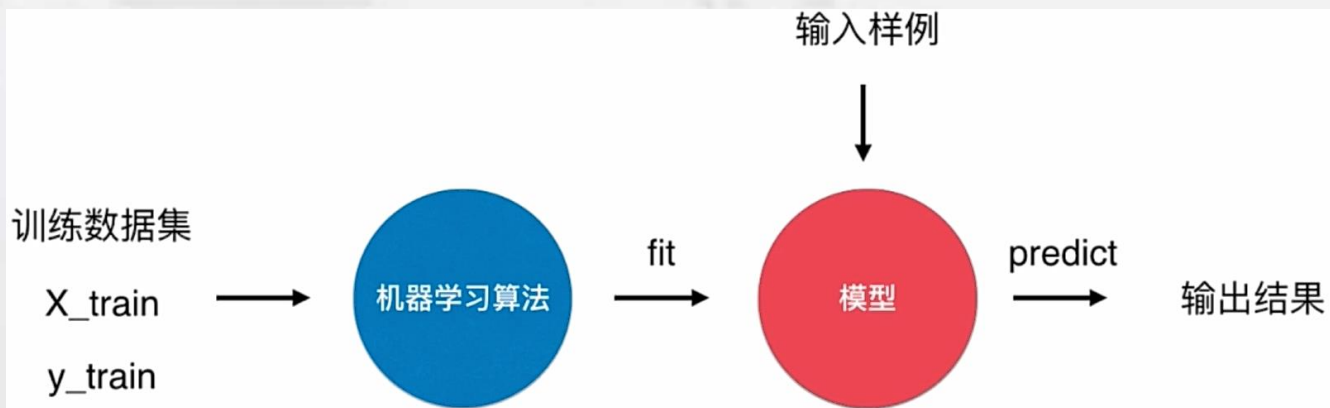
2

机器学习算法封装



输入样例可以直接送给训练数据集
训练过程就是 **fit** 拟合过程

在训练集中找到离输入样例最近的K个点，之后投票选择，投票数最高的那个标签就是所要的结果了



KNN是一个不需要训练过程的机器学习算法

所以KNN算法是非常特殊的，可以被认为是没有模型的算法，为了和其他算法统一，可以认为训练数据集就是模型本身

相对复杂的是预测过程

2

机器学习算法封装

2.2 研究scikit-learn机器学习库，使用学习库中封装好的KNN算法来解决分类任务

```
In [5]: 1 from sklearn.neighbors import KNeighborsClassifier
```

```
In [6]: 1 kNN_classifier = KNeighborsClassifier(n_neighbors=6)
```

```
In [7]: 1 kNN_classifier.fit(X_train, y_train)
```

```
Out[7]: KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',  
                             metric_params=None, n_jobs=1, n_neighbors=6, p=2,  
                             weights='uniform')
```

每一个机器学习算法，都会有**fit()**调用，传入两个参数，一是训练数据集对应的特征矩阵；二是标签向量。**fit()**返回值为KNN对象本身，可以不用其他变量接收这个返回值

因此KNN的模型可以理解为传入训练集参数的KNN对象

2

机器学习算法封装

使用scikit-learn中的KNN模型进行预测，需要通过实例kNN_classifier调用predict()方法，传入x样本，其中已知x为一维数组

```
20 x = np.array([8.093607318, 3.365731514])
```

```
In [8]: 1 kNN_classifier.predict(x)
```

```
/Users/yuanzhang/anaconda/lib/python3.6/site-packages/sklearn/utils/validation.py:395: DeprecationWarning: Passing 1d arrays as data is deprecated in 0.17 and will raise ValueError in 0.19. Reshape your data either using X.reshape(-1, 1) if your data has a single feature or X.reshape(1, -1) if it contains a single sample.
  DeprecationWarning)
```

```
Out[8]: array([1])
```

- 1、返回值是位于数组中的，我们要求的对象只有一个样本，所以返回的数组中，只有一个数，是1，和前面我们自己编写的算法结果是一致的。
- 2、警告信息表示，在predict()参数中传入一个一维数组方式已经过时，scikit-learn为了统一接口，所以在预测时，传入的参数应该是一个矩阵，即使数据只有一个，也要将数据转换成矩阵的形式。

2

机器学习算法封装

使用scikit-learn中的KNN模型进行预测，需要通过实例kNN_classifier调用predict()方法，传入x样本，其中已知x为一维数组

```
20 x = np.array([8.093607318, 3.365731514])
```

```
In [9]: 1 X_predict = x.reshape(1, -1)
```

一维数组调用reshape(1,-1)

第1个维度参数：1，表示一行数据，第2个维度参数：-1，表示让numpy自动决定第2个维度有多少，即1xn

```
In [11]: 1 X_predict
```

```
Out[11]: array([[ 8.09360732,  3.36573151]])
```

```
In [12]: 1 knn_classifier.predict(X_predict)
```

```
Out[12]: array([1])
```

```
In [13]: 1 y_predict = knn_classifier.predict(X_predict)
```

```
In [14]: 1 y_predict[0]
```

```
Out[14]: 1
```

2

机器学习算法封装

2.3 根据scikit-learn中KNN模型调用原理，设计自己的算法封装程序

```
import numpy as np
from math import sqrt
from collections import Counter
```

```
class KNNClassifier:
```

```
    def __init__(self, k):
        """初始化kNN分类器"""
        assert k >= 1, "k must be valid"
        self.k = k
        self._X_train = None
        self._y_train = None
```

```
    def fit(self, X_train, y_train):
        """根据训练数据集X_train和y_train训练kNN分类器"""
        assert X_train.shape[0] == y_train.shape[0], \
            "the size of X_train must be equal to the size of y_train"
        assert self.k <= X_train.shape[0], \
            "the size of X_train must be at least k."

        self._X_train = X_train
        self._y_train = y_train
        return self
```

这里fit()函数由于返回的是对象本身，所以仅完成赋值操作，此函数可以不返回，增加返回是遵循scikit-learn设计的原则

2

机器学习算法封装

2.3 根据scikit-learn中KNN模型调用原理，设计自己的算法封装程序

```
def predict(self, X_predict):  
    """给定待预测数据集X_predict，返回表示X_predict的结果向量"""  
    assert self._X_train is not None and self._y_train is not None, \  
        "must fit before predict!"  
    assert X_predict.shape[1] == self._X_train.shape[1], \  
        "the feature number of X_predict must be equal to X_train"  
  
    y_predict = [self._predict(x) for x in X_predict]  
    return np.array(y_predict)
```

此处predict(self, X_predict) 同样是依据scikit-learn原则，要求传入的参数是矩阵的形式；
X_predict.shape[1]：表示行即样本的个数可以不同，但列的个数，即特征的个数必须一致。
对于X_predict每一个样本，即每一行，进行预测，将预测结果放入一个向量中
_predict(x)：设计的私有函数，用于对一个向量进行的预测

2

机器学习算法封装

2.3 根据scikit-learn中KNN模型调用原理，设计自己的算法封装程序

```
def _predict(self, x):  
    """给定单个待预测数据x，返回x的预测结果值"""  
    assert x.shape[0] == self._X_train.shape[1], \  
        "the feature number of x must be equal to X_train"  
  
    distances = [sqrt(np.sum((x_train - x) ** 2))  
                 for x_train in self._X_train]  
    nearest = np.argsort(distances)  
  
    topK_y = [self._y_train[i] for i in nearest[:self.k]]  
    votes = Counter(topK_y)  
  
    return votes.most_common(1)[0][0]  
  
def __repr__(self):  
    return "KNN(k=%d)" % self.k
```

2

机器学习算法封装

2.4 调用设计好的算法封装程序

```
In [15]: 1 %run kNN/kNN.py

In [16]: 1 knn_clf = KNNClassifier(k=6)

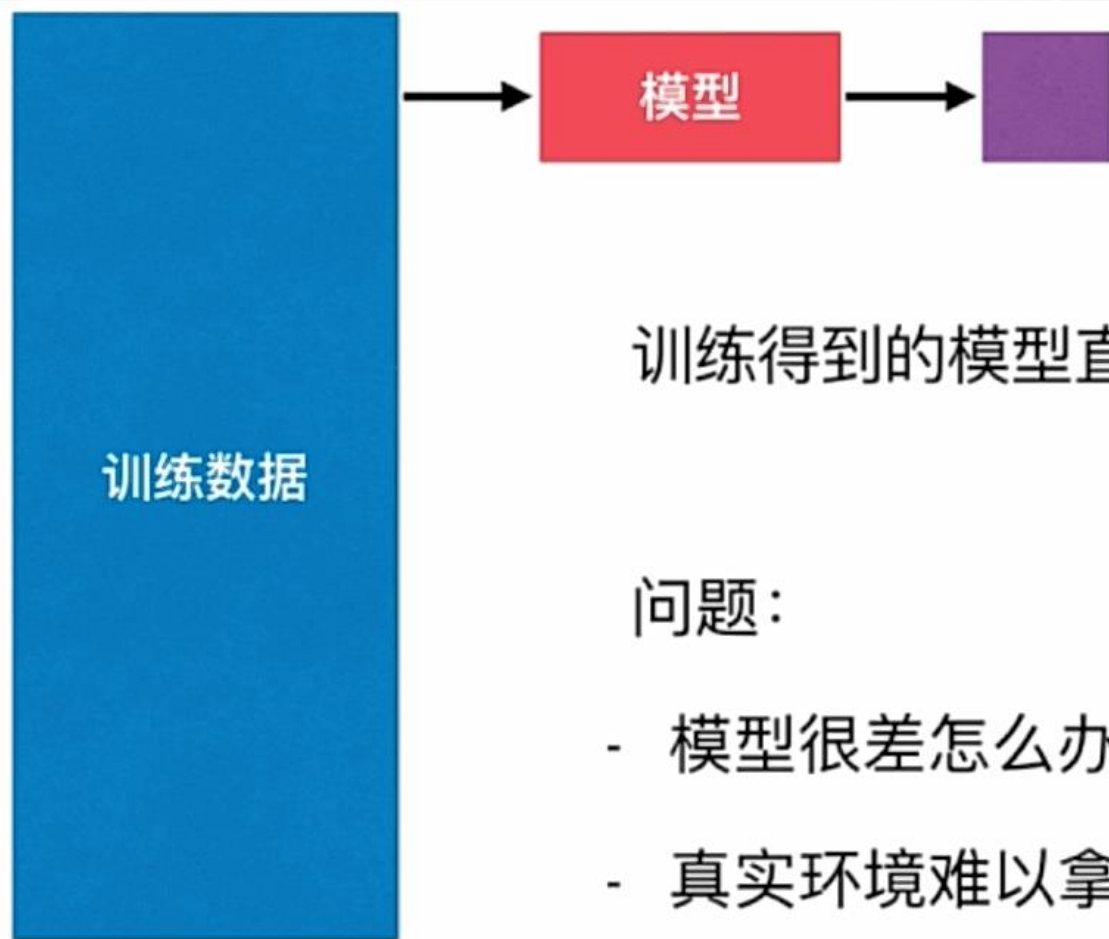
In [17]: 1 knn_clf.fit(X_train, y_train)
Out[17]: KNN(k=6)

In [18]: 1 y_predict = knn_clf.predict(X_predict)

In [19]: 1 y_predict
Out[19]: array([1])

In [20]: 1 y_predict[0]
Out[20]: 1
```

判断机器学习算法的性能



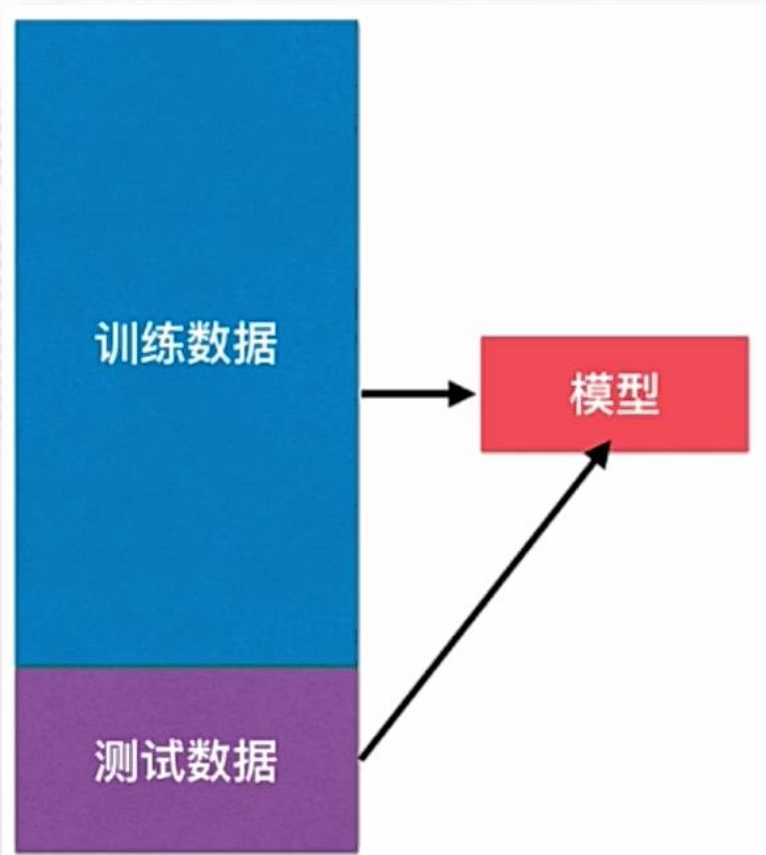
训练得到的模型直接在真实环境中使用。

问题：

- 模型很差怎么办？真实损失。
- 真实环境难以拿到真实label？

3

判断机器学习算法的性能



- 训练数据集用于训练模型
- 测试数据集用于评测训练出来的模型的效果好坏
- 在模型进入真实环境前改进模型

train test split

3

判断机器学习算法的性能

测试我们的算法

```
In [1]: 1 import numpy as np
        2 import matplotlib.pyplot as plt
        3 from sklearn import datasets
```

```
In [3]: 1 iris = datasets.load_iris()
```

```
In [4]: 1 X = iris.data
        2 y = iris.target
```

X: 数据集对应的特征矩阵
y: 结果标签对应的向量

```
In [5]: 1 X.shape
```

```
Out[5]: (150, 4)
```

```
In [6]: 1 y.shape
```

```
Out[6]: (150,)
```

train_test_split

```
In [7]: 1 y
Out[7]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

y向量中的值都是已经排好序的，如果按照这个顺序分组，训练出来的模型不是好的模型，而测试的数据又完全与训练样本不一致，这样就测试不出模型的好坏。因此需要进行乱序处理，由于X与y是对应的，二者应该同步。

方法一：X与y合并，之后进行shuffle()处理

方法二：对150个索引进行shuffle()处理

3

判断机器学习算法的性能

`permutation(100)`: 表示对0-99索引中的数据进行随机排序处理

```
In [8]: 1 shuffle_indexes = np.random.permutation(len(X))
```

```
In [9]: 1 shuffle_indexes
```

```
Out[9]: array([ 49,  74, 127,   2, 128, 100,  31, 114, 126,  28, 113,  23,  51,
        72, 129,  19, 111,   6,   1,  27, 105,   5,  39,  10,  52,  34,
        98,  88, 131, 134,  70, 133, 138,  58,  90,  54,  32, 139,  87,
       110, 123, 103,  82,  83,  12,  40, 107, 116,  44, 125,  66, 137,
        97,  56,  89, 136,  94,  84,  20, 146, 109,  14,  63,   9, 120,
        85,  33, 117,  53,  79,  45, 124,  81, 142,  69,  13,  75,  36,
         4,  64,  22,  86,  80,  16,   3,  26, 101,  65,   8,  71, 149,
       132, 141, 148,  62,  96, 119,  77,  38, 130,  95, 102,  48,  29,
        30,  24, 144,  35, 104,  59,  68,   7, 147,  91,  60,  15,  57,
       112,  21, 118,  93,  17, 121,  73,  37,  99,  55,  47,  78, 122,
        92,  43,  67, 115, 106,  42,  46, 135,  76, 145,  41,   0, 140,
        61, 143,  18,  11,  50, 108,  25])
```

```
In [10]: 1 test_ratio = 0.2          设定测试数据集的比例
         2 test_size = int(len(X) * test_ratio)
```

```
In [11]: 1 test_size
```

```
Out[11]: 30
```

permutation

permutation n. [数] 排列; [数] 置换

permutations n. [数] 排列 (permutation的复数)

permutation and combination n. 排列组合

permutation matrix [数] 置换矩阵

permutation test [统计] 排列检验; 排列测验

3

判断机器学习算法的性能

```
In [12]: 1 test_indexes = shuffle_indexes[:test_size]
          2 train_indexes = shuffle_indexes[test_size:]
```

```
In [13]: 1 X_train = X[train_indexes]
          2 y_train = y[train_indexes]
          3
          4 X_test = X[test_indexes]
          5 y_test = y[test_indexes]
```

```
In [14]: 1 print(X_train.shape)
          2 print(y_train.shape)
```

```
(120, 4)
(120,)
```

```
In [15]: 1 print(X_test.shape)
          2 print(y_test.shape)
```

```
(30, 4)
(30,)
```

由此完成train - test - split 的过程

3

判断机器学习算法的性能

model_selection.py

```
import numpy as np

def train_test_split(X, y, test_ratio=0.2, seed=None):
    """将数据 X 和 y 按照test_ratio分割成X_train, X_test, y_train, y_test"""
    assert X.shape[0] == y.shape[0], \
        "the size of X must be equal to the size of y"
    assert 0.0 <= test_ratio <= 1.0, \
        "test_ratio must be valid"

    if seed:
        np.random.seed(seed)

    shuffled_indexes = np.random.permutation(len(X))

    test_size = int(len(X) * test_ratio)
    test_indexes = shuffled_indexes[:test_size]
    train_indexes = shuffled_indexes[test_size:]

    X_train = X[train_indexes]
    y_train = y[train_indexes]

    X_test = X[test_indexes]
    y_test = y[test_indexes]
    return X_train, X_test, y_train, y_test
```

将前面算法逻辑封装成一个函数

测试机器学习算法的性能，实质上是为了帮忙我们选择一个更好的模型，若发现模型不够好，则改进算法，选择其他的模型

3

判断机器学习算法的性能

使用我们的算法

```
In [16]: 1 from playML.model_selection import train_test_split
```

```
In [17]: 1 X_train, X_test, y_train, y_test = train_test_split(X, y)
```

```
In [18]: 1 print(X_train.shape)  
2 print(y_train.shape)
```

```
(120, 4)  
(120,)
```

```
In [19]: 1 print(X_test.shape)  
2 print(y_test.shape)
```

```
(30, 4)  
(30,)
```

```
In [20]: 1 from playML.kNN import KNNClassifier
```

```
In [27]: 1 my_knn_clf = KNNClassifier(k=3)
```

```
In [28]: 1 my_knn_clf.fit(X_train, y_train)
```

```
Out[28]: KNN(k=3)
```


3

判断机器学习算法的性能

```
In [31]: 1 y_predict = my_knn_clf.predict(X_test)
```

```
In [32]: 1 y_predict
```

```
Out[32]: array([2, 2, 0, 1, 0, 1, 1, 0, 2, 0, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1, 2,
0,
1, 2, 0, 2, 2, 1, 0])
```

包含30个X_test对应的结果
与测试集中标签值对比

```
In [33]: 1 y_test
```

```
Out[33]: array([2, 2, 0, 1, 0, 1, 1, 0, 2, 0, 0, 1, 2, 1, 1, 1, 2, 0, 1, 1, 1, 2,
0,
1, 2, 0, 2, 2, 1, 0])
```

```
In [34]: 1 sum([1 if y_predict[i] == y_test[i] else 0 for i in range(len(y_test))])
```

```
Out[34]: 29
```

```
In [35]: 1 y_test.shape
```

```
Out[35]: (30,)
```

```
In [36]: 1 sum([1 if y_predict[i] == y_test[i] else 0 for i in range(len(y_test))]) / len(y_test)
```

```
Out[36]: 0.9666666666666667
```

```
In [49]: 1 sum(y_predict == y_test)
```

```
Out[49]: 29
```

```
In [50]: 1 sum(y_predict == y_test) / len(y_test)
```

```
Out[50]: 0.9666666666666667
```

分类准确度

3

判断机器学习算法的性能

sklearn机器学习库中所封装的train_test_split

```
In [37]: 1 from sklearn.model_selection import train_test_split
```

```
In [38]: 1 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2 random_state=666
```

此处整个函数接口的设计是一样的，将原始数据集拆分成训练数据集X 和测试数据集y，其中test_size表示设置的测试数据集的比例，若不传值，默认是0.2

由于train_test_split()函数带有随机数的性质，所以参数可设置随机种子random_state = 666

```
In [39]: 1 print(X_train.shape)
          2 print(y_train.shape)
```

```
(120, 4)
(120,)
```

```
In [40]: 1 print(X_test.shape)
          2 print(y_test.shape)
```

```
(30, 4)
(30,)
```

后面学习使用train_test_split来进行模型的选择

3

判断机器学习算法的性能

用一个新的数据集再模拟一下前面的过程
导入scikit-learn中的手写数字数据集

```
In [2]: 1 import numpy as np
        2 import matplotlib
        3 import matplotlib.pyplot as plt
        4 from sklearn import datasets
```

```
In [3]: 1 digits = datasets.load_digits()
```

```
In [4]: 1 digits.keys()
```

```
Out[4]: dict_keys(['data', 'target', 'target_names', 'images', 'DESCR'])
```

创建数据集对象digits

digits可以看作是一个字典

所以调用keys()可以查看一下键值属性，与iris数据集是一致的

3

判断机器学习算法的性能

```
In [6]: 1 print(digits.DESCR)
```

```
Optical Recognition of Handwritten Digits Data Set
```

```
=====
```

```
Notes
```

```
-----
```

```
Data Set Characteristics:
```

```
:Number of Instances: 5620
```

```
:Number of Attributes: 64
```

```
:Attribute Information: 8x8 image of integer pixels in the range 0..1
```

```
6.
```

```
:Missing Attribute Values: None
```

```
:Creator: E. Alpaydin (alpaydin '@' boun.edu.tr)
```

```
:Date: July; 1998
```

```
This is a copy of the test set of the UCI ML hand-written digits datasets
```

```
http://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits
```

```
The data set contains images of hand-written digits: 10 classes where  
each class refers to a digit.
```

这是一个简化的
副本，其实只有
2000多样本

0-9 数字

3

判断机器学习算法的性能

```
In [7]: 1 X = digits.data
        2 X.shape
```

```
Out[7]: (1797, 64)
```

```
In [8]: 1 y = digits.target
```

```
In [9]: 1 y.shape
```

```
Out[9]: (1797,)
```

X: 数据集特征值矩阵

y: 数据集标签向量

```
In [10]: 1 digits.target_names
```

```
Out[10]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

取前100个标签值，与iris有些不同

```
In [11]: 1 y[:100]
```

```
Out[11]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1,
                2,
                3, 4, 5, 6, 7, 8, 9, 0, 9, 5, 5, 6, 5, 0, 9, 8, 9, 8, 4, 1, 7, 7,
                3,
                5, 1, 0, 0, 2, 2, 7, 8, 2, 0, 1, 2, 6, 3, 3, 7, 3, 3, 4, 6, 6, 6,
                4,
                9, 1, 5, 0, 9, 5, 2, 8, 2, 0, 0, 1, 7, 6, 3, 2, 1, 7, 4, 6, 3, 1,
                3,
                9, 1, 7, 6, 8, 4, 3, 1])
```


3

判断机器学习算法的性能

取X前10个元素，表示前10个样本，即前10行

```
In [12]: 1 X[:10]
```

```
Out[12]: array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.,  0.,  0., 13.,
 15., 10., 15.,  5.,  0.,  0.,  3., 15.,  2.,  0., 11.,
  8.,  0.,  0.,  4., 12.,  0.,  0.,  8.,  8.,  0.,  0.,
  5.,  8.,  0.,  0.,  9.,  8.,  0.,  0.,  4., 11.,  0.,
  1., 12.,  7.,  0.,  0.,  2., 14.,  5., 10., 12.,  0.,
  0.,  0.,  0.,  6., 13., 10.,  0.,  0.,  0.],
 [ 0.,  0.,  0., 12., 13.,  5.,  0.,  0.,  0.,  0.,  0.,
 11., 16.,  9.,  0.,  0.,  0.,  0.,  3., 15., 16.,  6.,
  0.,  0.,  0.,  7., 15., 16., 16.,  2.,  0.,  0.,  0.,
  0.,  1., 16., 16.,  3.,  0.,  0.,  0.,  0.,  1., 16.,
 16.,  6.,  0.,  0.,  0.,  0.,  1., 16., 16.,  6.,  0.,
  0.,  0.,  0.,  0., 11., 16., 10.,  0.,  0.],
 [ 0.,  0.,  0.,  4., 15., 12.,  0.,  0.,  0.,  0.,  3.,
 16., 15., 14.,  0.,  0.,  0.,  0.,  8., 13.,  8., 16.,
  0.,  0.,  0.,  0.,  1.,  6., 15., 11.,  0.,  0.,  0.,
  1.,  8., 13., 15.,  1.,  0.,  0.,  0.,  9., 16., 16.,
  5.,  0.,  0.,  0.,  0.,  3., 13., 16., 16., 11.,  5.,
  0.,  0.,  0.,  0.,  3., 11., 16.,  9.,  0.],
 [ 0.,  0.,  7., 15., 13.,  1.,  0.,  0.,  0.,  8., 13.,
  6., 15.,  4.,  0.,  0.,  0.,  2.,  1., 13., 13.,  0.,
  0.,  0.,  0.,  0.,  2., 15., 11.,  1.,  0.,  0.,  0.,
  0.,  0.,  1., 12., 12.,  1.,  0.,  0.,  0.,  0.,  0.,
  1., 10.,  8.,  0.,  0.,  0.,  8.,  4.,  5., 14.,  9.,
  0.,  0.,  0.,  7., 13., 13.,  9.,  0.,  0.]
```

3

判断机器学习算法的性能

```
In [13]: 1 some_digit = X[666]
```

取出一个样本进行观察，y标签值为0，代表第666行表示数字0

```
In [14]: 1 y[666]
```

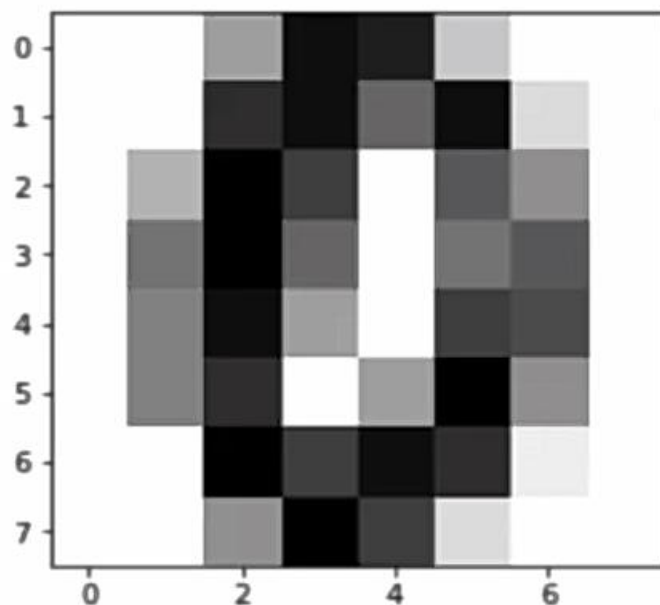
```
Out[14]: 0
```

```
In [15]: 1 some_digit_image = some_digit.reshape(8, 8)
2 plt.imshow(some_digit_image, cmap = matplotlib.cm.binary)
3 plt.show()
```

生成8X8的矩阵

调用imshow()进行可视化

可以看到数字0



3

判断机器学习算法的性能

```
In [16]: 1 from playML.model_selection import train_test_split
```

```
In [17]: 1 X_train, X_test, y_train, y_test = train_test_split(X, y, test_ratio=0.2)
```

加载进playML包中KNN脚本中的KNNClassifier类

```
In [18]: 1 from playML.kNN import KNNClassifier
```

创建自己的KNNClassifier，设 $k = 3$ ，调用predict()进行预测

```
In [20]: 1 my_knn_clf.fit(X_train, y_train)
```

```
Out[20]: KNN(k=3)
```

```
In [21]: 1 y_predict = my_knn_clf.predict(X_test)
```

```
In [22]: 1 y_predict
```

```
Out[22]: array([3, 3, 2, 0, 4, 8, 2, 5, 3, 7, 7, 3, 0, 4, 9, 1, 5, 2, 4, 2, 7, 4,
6,
        6, 1, 7, 4, 4, 7, 6, 0, 3, 6, 6, 9, 0, 1, 9, 3, 8, 3, 2, 8, 2, 5,
9,
```

下面演示分类以及计算分类准确度的流程

3

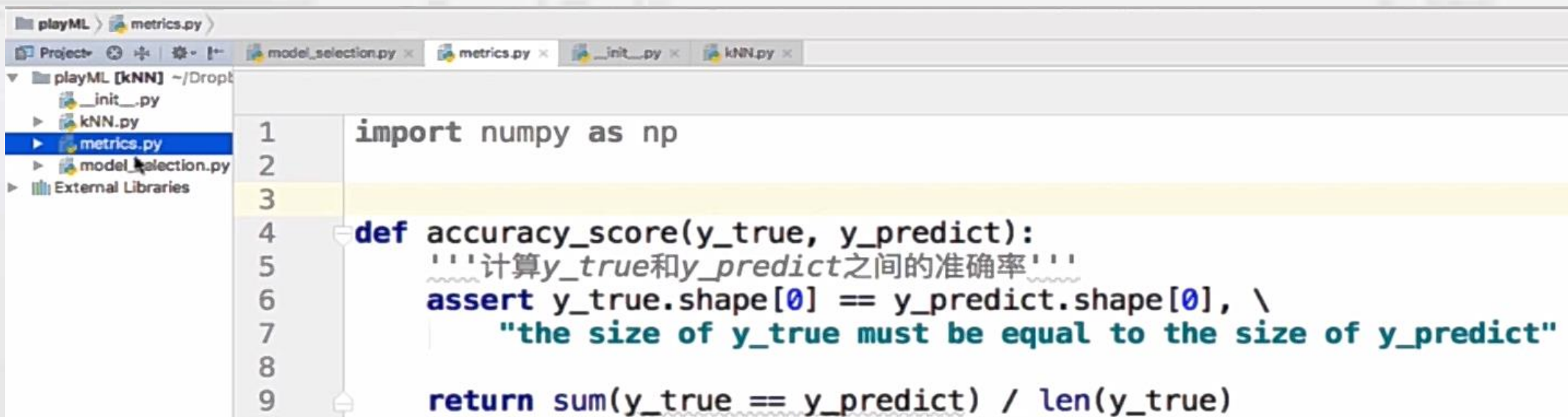
判断机器学习算法的性能

预测成功的数字再除以进行预测的总数字，结果就是最终的预测结果的准确度

```
In [24]: 1 sum(y_predict == y_test) / len(y_test)
Out[24]: 0.9888579387186629
```

对比训练集样本的预测结果和测试集中的数据，求得预测结果的准确度，是一种常规操作

在playML包中对分类算法的准确率进行封装



```
playML > metrics.py
Project: model_selection.py x metrics.py x __init__.py x kNN.py x
playML [kNN] ~/Dropbox
├── __init__.py
├── kNN.py
├── metrics.py
├── model_selection.py
└── External Libraries
1 import numpy as np
2
3
4 def accuracy_score(y_true, y_predict):
5     '''计算y_true和y_predict之间的准确率'''
6     assert y_true.shape[0] == y_predict.shape[0], \
7         "the size of y_true must be equal to the size of y_predict"
8
9     return sum(y_true == y_predict) / len(y_true)
```

3

判断机器学习算法的性能

预测成功的数字再除以进行预测的总数字，结果就是最终的预测结果的准确度

```
In [24]: 1 sum(y_predict == y_test) / len(y_test)
```

```
Out[24]: 0.9888579387186629
```

```
In [25]: 1 from playML.metrics import accuracy_score  
2  
3 accuracy_score(y_test, y_predict)
```

```
Out[25]: 0.9888579387186629
```

以后可以直接调用封装好的`accuracy_score()`函数来计算预测结果的准确度

3

判断机器学习算法的性能

有时我们对预测值是多少不感兴趣，只对准确率感兴趣
因此，需要在KNN算法模型中进一步封装一个函数

```
4 from .metrics import accuracy_score

51 def score(self, X_test, y_test):
52     """根据测试数据集 X_test 和 y_test 确定当前模型的准确度"""
53     y_predict = self.predict(X_test)
54     return accuracy_score(y_test, y_predict)
```

KNN算法模型进行了调整，因此需要重新编译运行，最后结果如下，与前面的计算结果是一样的

```
In [24]: 1 my_knn_clf.score(X_test, y_test)
Out[24]: 0.99442896935933145
```

3

判断机器学习算法的性能

再使用scikit-learn完成整个的过程，通过比较，逐步了解scikit-learn机器学习的使用方法

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=666)
```

KNN算法模型进行了调整，因此需要重新编译运行，最后结果如下，与前面的计算结果是一样的

```
In [26]: 1 from sklearn.neighbors import KNeighborsClassifier
          2
          3 knn_clf = KNeighborsClassifier(n_neighbors=3)
```

```
In [28]: 1 knn_clf.fit(X_train, y_train)
```

```
Out[28]: KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                               metric_params=None, n_jobs=1, n_neighbors=3, p=2,
                               weights='uniform')
```

```
In [29]: 1 y_predict = knn_clf.predict(X_test)
```

```
In [30]: 1 from sklearn.metrics import accuracy_score
          2
          3 accuracy_score(y_test, y_predict)
```

```
Out[30]: 0.98888888888888893
```

n_neighbors参数
如何确定？

4

超参数

超参数

```
In [1]: 1 import numpy as np
        2 from sklearn import datasets
```

```
In [2]: 1 digits = datasets.load_digits()
        2 X = digits.data
        3 y = digits.target
```

```
In [3]: 1 from sklearn.model_selection import train_test_split
        2
        3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
In [4]: 1 from sklearn.neighbors import KNeighborsClassifier
        2
        3 knn_clf = KNeighborsClassifier(n_neighbors=3) 传什么样的值最好?
        4 knn_clf.fit(X_train, y_train)
        5 knn_clf.score(X_test, y_test)
```

```
Out[4]: 0.98888888888888893
```

4

超参数

超参数和模型参数

超参数：在算法运行前需要决定的参数

模型参数：算法过程中学习的参数

调参，调的就是这些超参数

KNN算法没有模型参数

KNN算法中的K是典型的超参数

寻找好的超参数

- 领域知识
- 经验数值
- 实验搜索

4

超参数

寻找最好的k

```
In [13]: 1 best_score = 0.0
          2 best_k = -1
          3 for k in range(1, 11):
          4     knn_clf = KNeighborsClassifier(n_neighbors=k)
          5     knn_clf.fit(X_train, y_train)
          6     score = knn_clf.score(X_test, y_test)
          7     if score > best_score:
          8         best_k = k
          9         best_score = score
          10
          11 print("best_k =", best_k)
          12 print("best_score =", best_score)
```

```
best_k = 4
best_score = 0.991666666667
```

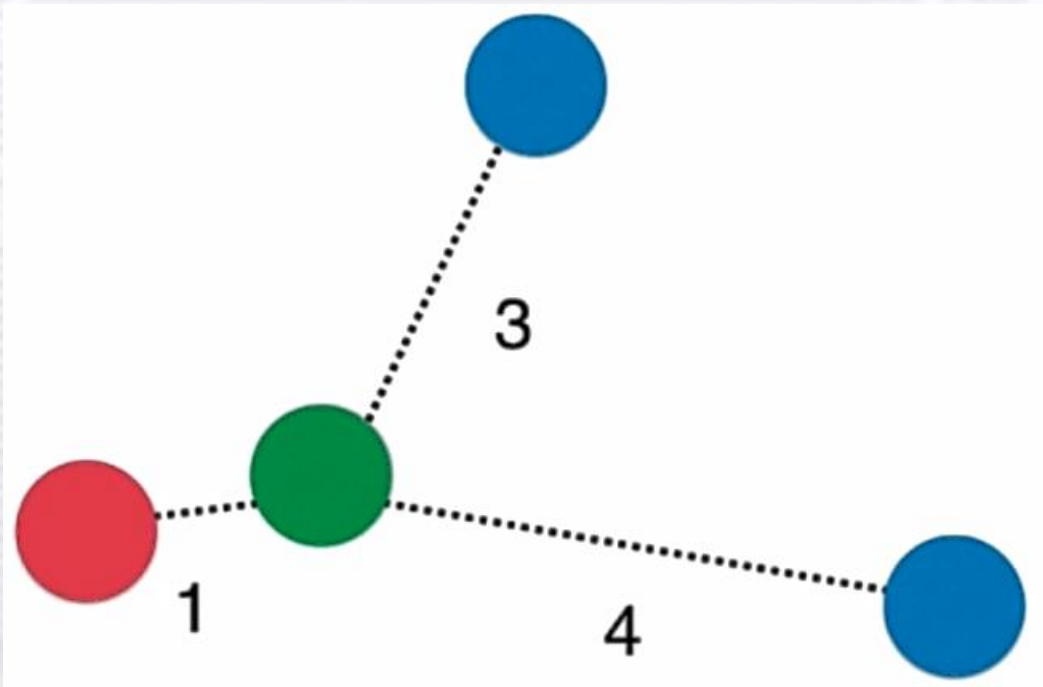
KNN算法，如何寻找最好的K

即为调参过程

若 $k = 10$, 则有必要在11-20范围进行搜索，即找到的值在边界上，就拓展搜索范围，对边界外围再进行查找

4

超参数

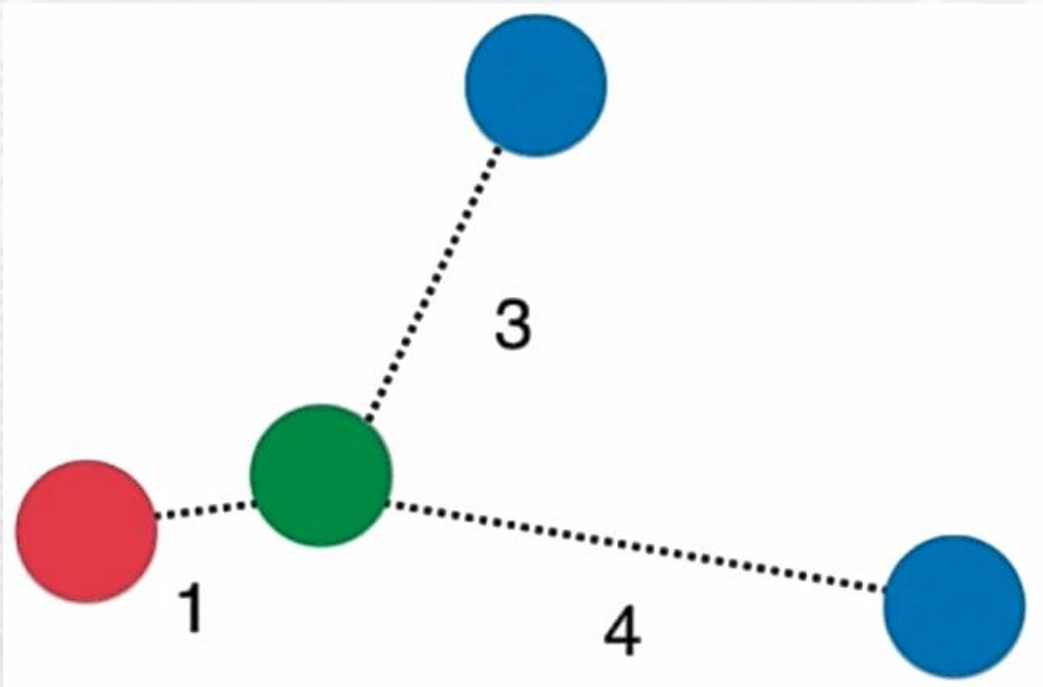


普通的KNN算法，蓝色获胜

考虑到距离作为权重

4

超参数

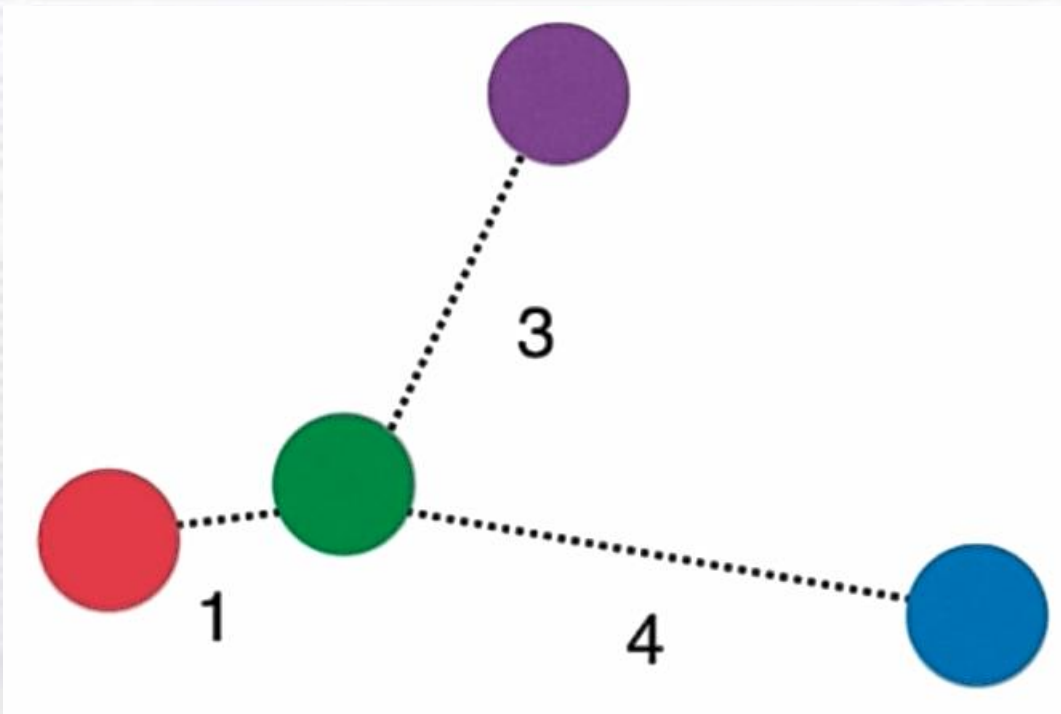


普通的KNN算法，蓝色获胜

考虑距离：红色：1 蓝色： $\frac{1}{3} + \frac{1}{4} = \frac{7}{12}$ 红色胜

4

超参数



普通的KNN算法，存在 **平票** 的问题

考虑距离之后，可以解决平票问题

4

超参数

scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html



Home Installation Documentation Examples

Google Custom Search

Search x

Previous
sklearn.neighbors.KNeighborsClassifier

Next
sklearn.neighbors.KNeighborsClassifier

Up
API
Reference

scikit-learn v0.19.1
Other versions

Please [cite us](#) if you use
the software.

`sklearn.neighbors.KNeighborsClassifier`

Examples using

`sklearn.neighbors.KNeighborsClassifier`

sklearn.neighbors.KNeighborsClassifier

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, weights='uniform', algorithm='auto',  
leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=1, **kwargs)
```

[\[source\]](#)

Classifier implementing the k-nearest neighbors vote.

Read more in the [User Guide](#).

Parameters: `n_neighbors` : int, optional (default = 5)

Number of neighbors to use by default for `kneighbors` queries.

weights : str or callable, optional (default = 'uniform')

weight function used in prediction. Possible values:

- 'uniform' : uniform weights. All points in each neighborhood are weighted equally.
- 'distance' : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
- [callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

algorithm : {'auto', 'ball_tree', 'kd_tree', 'brute'}, optional

4

超参数

考虑距离或不考虑距离

```
In [15]: 1 best_method = ""
2 best_score = 0.0
3 best_k = -1
4 for method in ["uniform", "distance"]:
5     for k in range(1, 11):
6         knn_clf = KNeighborsClassifier(n_neighbors=k, weights=method)
7         knn_clf.fit(X_train, y_train)
8         score = knn_clf.score(X_test, y_test)
9         if score > best_score:
10             best_k = k
11             best_score = score
12             best_method = method
13
14 print("best_method =", best_method)
15 print("best_k =", best_k)
16 print("best_score =", best_score)
```

```
best_method = uniform
best_k = 4
best_score = 0.991666666667
```

weights : 缺省是' uniform' 表示不考虑距离

4

超参数

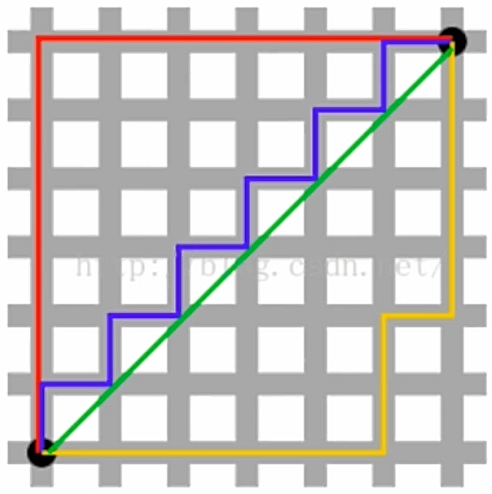
更多关于距离的定义

欧拉距离

$$\sqrt{\sum_{i=1}^n (X_i^{(a)} - X_i^{(b)})^2}$$

曼哈顿距离

$$\sum_{i=1}^n |X_i^{(a)} - X_i^{(b)}|$$



$$\left(\sum_{i=1}^n |X_i^{(a)} - X_i^{(b)}| \right)^{\frac{1}{1}}$$

$$\left(\sum_{i=1}^n |X_i^{(a)} - X_i^{(b)}|^2 \right)^{\frac{1}{2}}$$

$$\left(\sum_{i=1}^n |X_i^{(a)} - X_i^{(b)}|^p \right)^{\frac{1}{p}}$$

4

超参数

更多关于距离的定义

$$\left(\sum_{i=1}^n |X_i^{(a)} - X_i^{(b)}| \right)^{\frac{1}{1}}$$

$$\left(\sum_{i=1}^n |X_i^{(a)} - X_i^{(b)}|^2 \right)^{\frac{1}{2}}$$

$$\left(\sum_{i=1}^n |X_i^{(a)} - X_i^{(b)}|^p \right)^{\frac{1}{p}}$$

明可夫斯基距离

Minkowski Distance

$$\left(\sum_{i=1}^n |X_i^{(a)} - X_i^{(b)}|^p \right)^{\frac{1}{p}}$$

p : integer, optional (default = 2)

Power parameter for the **Minkowski** metric. When $p = 1$, this is equivalent to using `manhattan_distance (l1)`, and `euclidean_distance (l2)` for $p = 2$. For arbitrary p , `minkowski_distance (l_p)` is used.

获得一个超参数P

4

超参数

搜索明可夫斯基距离相应的p

In [17]:

```
1 %%time
2
3 best_p = -1
4 best_score = 0.0
5 best_k = -1
6
7 for k in range(1, 11):
8     for p in range(1, 6):
9         knn_clf = KNeighborsClassifier(n_neighbors=k, weights="distance", p=p)
10        knn_clf.fit(X_train, y_train)
11        score = knn_clf.score(X_test, y_test)
12        if score > best_score:
13            best_k = k
14            best_score = score
15            best_p = p
16
17 print("best_p =", best_p)
18 print("best_k =", best_k)
19 print("best_score =", best_score)
```

best_p = 2

best_k = 3

best_score = 0.98888888888889

CPU times: user 28.9 s, sys: 1.27 s, total: 30.2 s

Wall time: 36 s

4

超参数

为了找到一个更好的模型

网格搜索

```
In [1]: 1 import numpy as np
        2 from sklearn import datasets
```

```
In [2]: 1 digits = datasets.load_digits()
        2 X = digits.data
        3 y = digits.target
```

```
In [3]: 1 _selection import train_test_split
        2
        3 _train, y_test = train_test_split(X, y, test_size=0.2, random_state=666)
```

```
In [4]: 1 from sklearn.neighbors import KNeighborsClassifier
        2
        3 sk_knn_clf = KNeighborsClassifier(n_neighbors=4, weights="uniform")
        4 sk_knn_clf.fit(X_train, y_train)
        5 sk_knn_clf.score(X_test, y_test)
```

```
Out[4]: 0.9916666666666667
```

4

超参数

Grid Search

```
In [5]: 1 param_grid = [  
2         {  
3             'weights': ['uniform'],  
4             'n_neighbors': [i for i in range(1, 11)]  
5         },  
6         {  
7             'weights': ['distance'],  
8             'n_neighbors': [i for i in range(1, 11)],  
9             'p': [i for i in range(1, 6)]  
10        }  
11    ]
```

```
In [6]: 1 knn_clf = KNeighborsClassifier()
```

```
In [7]: 1 from sklearn.model_selection import GridSearchCV  
2  
3 grid_search = GridSearchCV(knn_clf, param_grid)
```


4

超参数

```
In [7]: 1 from sklearn.model_selection import GridSearchCV
        2
        3 grid_search = GridSearchCV(knn_clf, param_grid)
```

```
In [8]: 1 %%time
        2 grid_search.fit(X_train, y_train)
```

CPU times: user 3min 32s, sys: 5.9 s, total: 3min 38s
Wall time: 3min 55s

```
Out[8]: GridSearchCV(cv=None, error_score='raise',
                    estimator=KNeighborsClassifier(algorithm='auto', leaf_size=30, met
                    ric='minkowski',
                    metric_params=None, n_jobs=1, n_neighbors=5, p=2,
                    weights='uniform'),
                    fit_params={}, iid=True, n_jobs=1,
                    param_grid=[{'weights': ['uniform'], 'n_neighbors': [1, 2, 3, 4,
                    5, 6, 7, 8, 9, 10]}, {'weights': ['distance'], 'n_neighbors': [1, 2, 3,
                    4, 5, 6, 7, 8, 9, 10], 'p': [1, 2, 3, 4, 5]}],
                    pre_dispatch='2*n_jobs', refit=True, return_train_score=True,
                    scoring=None, verbose=0)
```

4

超参数

搜索到的参数值

```
In [10]: 1 grid_search.best_estimator_
```

```
Out[10]: KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',  
                             metric_params=None, n_jobs=1, n_neighbors=3, p=3,  
                             weights='distance')
```

准确度

```
In [11]: 1 grid_search.best_score_
```

```
Out[11]: 0.98538622129436326
```

```
In [12]: 1 grid_search.best_params_
```

```
Out[12]: {'n_neighbors': 3, 'p': 3, 'weights': 'distance'}
```

4

超参数

```
In [13]: 1 knn_clf = grid_search.best_estimator_
```

```
In [14]: 1 knn_clf.predict(X_test)
```

```
Out[14]: array([8, 1, 3, 4, 4, 0, 7, 0, 8, 0, 4, 6, 1, 1, 2, 0, 1, 6, 7, 3, 3, 6,
5,
          2, 9, 4, 0, 2, 0, 3, 0, 8, 7, 2, 3, 5, 1, 3, 1, 5, 8, 6, 2, 6, 3,
1,
          3, 0, 0, 4, 9, 9, 2, 8, 7, 0, 5, 4, 0, 9, 5, 5, 8, 7, 4, 2, 8, 8,
7,
          5, 4, 3, 0, 2, 7, 2, 1, 2, 4, 0, 9, 0, 6, 6, 2, 0, 0, 5, 4, 4, 3,
1,
          3, 8, 6, 4, 4, 7, 5, 6, 8, 4, 8, 4, 6, 9, 7, 7, 0, 8, 8, 3, 9, 7,
1,
          8, 4, 2, 7, 0, 0, 4, 9, 6, 7, 3, 4, 6, 4, 8, 4, 7, 2, 6, 9, 5, 8,
7,
          2, 5, 5, 9, 7, 9, 3, 1, 9, 4, 4, 1, 5, 1, 6, 4, 4, 8, 1, 6, 2, 5,
2,
          1, 4, 4, 3, 9, 4, 0, 6, 0, 8, 3, 8, 7, 3, 0, 3, 0, 5, 9, 2, 7, 1,
```

```
In [15]: 1 knn_clf.score(X_test, y_test)
```

```
Out[15]: 0.98333333333333328
```

4

超参数

```
In [*]: 1 %%time
        2 grid_search = GridSearchCV(knn_clf, param_grid, n_jobs=-1, verbose=2)
        3 grid_search.fit(X_train, y_train)
```

Fitting 3 folds for each of 60 candidates, totalling 180 fits

```
[CV] n_neighbors=1, weights=uniform .....
[CV] n_neighbors=1, weights=uniform .....
[CV] n_neighbors=1, weights=uniform .....
[CV] n_neighbors=2, weights=uniform .....
[CV] ..... n_neighbors=1, weights=uniform, total= 1.2s
[CV] n_neighbors=2, weights=uniform .....
[CV] ..... n_neighbors=1, weights=uniform, total= 1.4s
[CV] n_neighbors=2, weights=uniform .....
[CV] ..... n_neighbors=1, weights=uniform, total= 1.5s
[CV] n_neighbors=3, weights=uniform .....
```

4

超参数

更多的距离定义

- 向量空间余弦相似度 Cosine Similarity
- 调整余弦相似度 Adjusted Cosine Similarity
- 皮尔森相关系数 Pearson Correlation Coefficient
- Jaccard相似系数 Jaccard Coefficient



谢谢